TestWarden

Weekly Agile Sprint Plan and Task Distribution

The 8 team members are reallocated every week. No permanent ownership is assumed.

How to use this plan

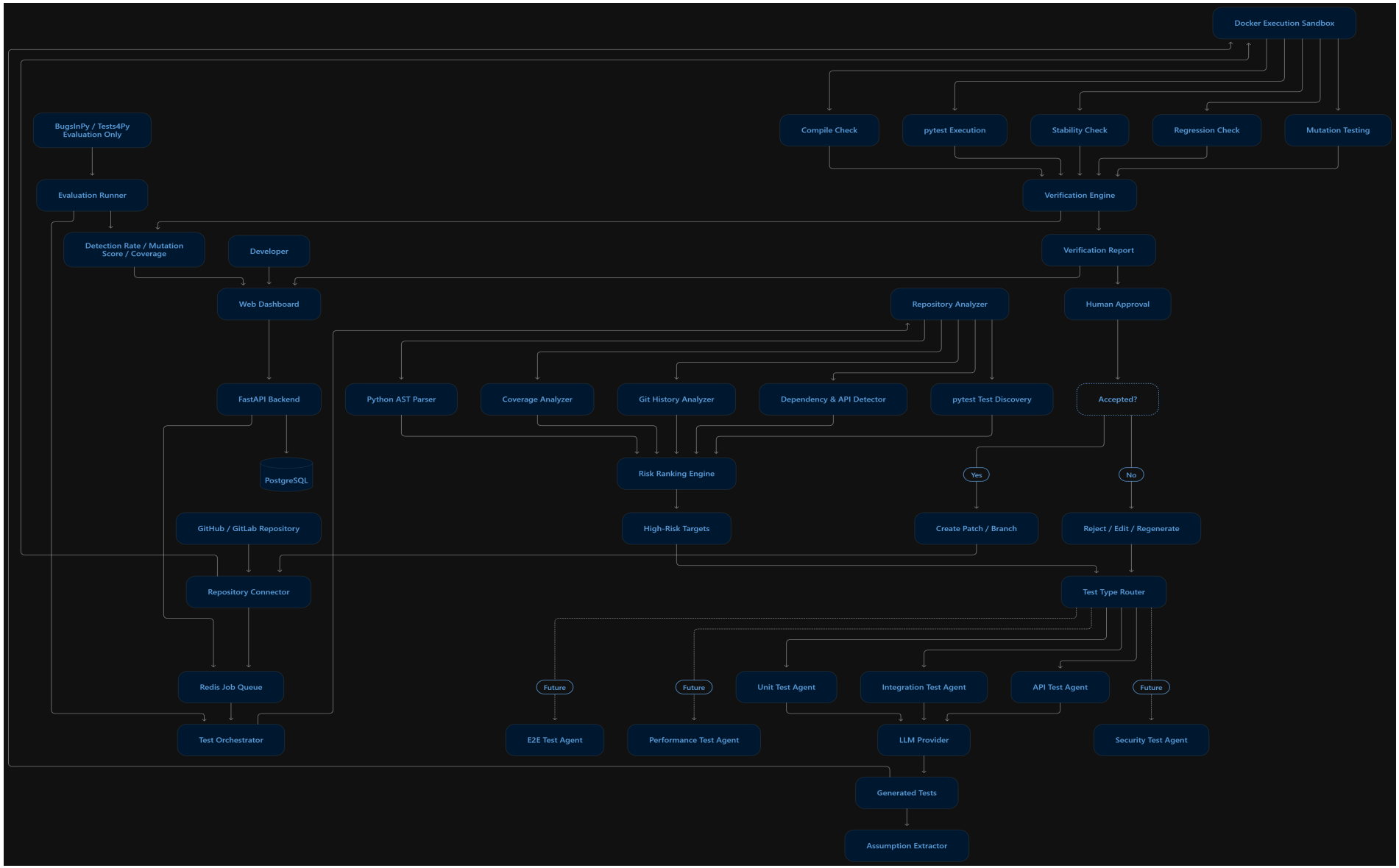
Each week is one Sprint. Every member receives one concrete task. The team reviews the end product at the end of the week. Tasks can be adjusted during planning, but the weekly deliverable must remain clear.

Team labels

M1 to M8 are temporary labels. Replace them with the actual team member names before sharing the plan.

System Architecture

The architecture below shows the complete TestWarden flow from repository connection to verification and approval.



Important MVP note: Unit Test Agent is implemented first. Integration and API Agents follow. Security, E2E, and Performance Agents remain future extensions.

Project File Architecture

The file structure below is the target structure. The weekly tables identify exactly which files or artifacts are touched by each task.

Folder / file area	Purpose
app/api/	FastAPI routes for repositories, analysis, generation, verification, approval, and evaluation.
app/models/	PostgreSQL database models for repositories, runs, risks, tests, verification, and approvals.
app/schemas/	Request and response contracts shared between the frontend, backend, workers, and services.
app/analyzers/	Python AST, pytest discovery, coverage, complexity, Git history, dependencies, and API detection.
app/agents/	Unit Test Agent, Integration Test Agent, API Test Agent, and the test type router.
app/ai/	LLM client, context builders, prompts, assumption extraction, and test repair.
app/execution/	Docker runner, dependency installation, pytest runner, logs, timeouts, and resource limits.
app/verification/	Compile checks, pytest checks, stability, regression, mutation execution, and verification engine.
app/orchestration/	Workflow states, job manager, and coordination of analysis, generation, and verification.
app/evaluation/	BugsInPy/Tests4Py adapters, benchmark runner, metrics, and evaluation reports.
workers/	Redis/Celery background tasks for long-running analysis, generation, verification, and evaluation jobs.
frontend/	Dashboard pages and components for risk, generated tests, reports, approval, and evaluation.
tests/	Unit, integration, API, regression, and end-to-end tests for TestWarden itself.
docker/	Sandbox image, benchmark image, test environments, and execution scripts.
benchmarks/	Selected Python repositories and benchmark subjects used for evaluation.
reports/	Generated experiment results, metrics tables, and final evaluation outputs.
docs/	Architecture, API, testing strategy, security, demo script, and project documentation.

Rule: every task must update the relevant file, add or update tests, and leave demo evidence for the sprint review.

Week 1 - Project Setup

End product: running project skeleton and team contracts.

Member	Task for this week	Files / artifacts	Done when
M1	Create Git repository, branches, issue labels, and contribution rules.	.git/; CONTRIBUTING.md; .github/	Code + test + demo evidence
M2	Create FastAPI project structure and health endpoint.	app/main.py; app/api/health.py	Code + test + demo evidence
M3	Create PostgreSQL schema draft and environment configuration.	app/core/config.py; .env.example; migrations/	Code + test + demo evidence
M4	Create basic LLM client prototype and safe configuration.	app/ai/llm_client.py; app/ai/config.py	Code + test + demo evidence
M5	Select two sample Python/pytest repositories and document setup.	benchmarks/samples/; docs/sample-repositories.md	Code + test + demo evidence
M6	Create Docker sandbox that runs a Python command.	docker/sandbox/Dockerfile; docker/sandbox/run.py	Code + test + demo evidence
M7	Define standard JSON result formats for jobs and test runs.	app/schemas/jobs.py; app/schemas/results.py	Code + test + demo evidence
M8	Write initial README, architecture notes, and a smoke-test checklist.	README.md; docs/architecture.md; docs/smoke-test.md	Code + test + demo evidence

Week 2 - Repository Connection

End product: a user can submit a Git URL and prepare a local project.

Member	Task for this week	Files / artifacts	Done when
M1	Implement Git clone and branch/commit selection.	app/services/repository_connector.py; app/api/routes_repositories.py	Code + test + demo evidence
M2	Detect Python files and project root.	app/analyzers/project_detector.py	Code + test + demo evidence
M3	Detect pytest, pyproject.toml, requirements, and test folders.	app/analyzers/framework_detector.py; pyproject.toml	Code + test + demo evidence
M4	Build repository context summary for the AI.	app/ai/context_builder.py	Code + test + demo evidence
M5	Define unsupported repository cases and sample error messages.	app/schemas/support.py; docs/supported-projects.md	Code + test + demo evidence
M6	Install dependencies inside Docker with timeout handling.	docker/sandbox/install_dependencies.sh	Code + test + demo evidence
M7	Parse clone/build/run logs into structured job results.	app/execution/log_parser.py; app/schemas/execution.py	Code + test + demo evidence
M8	Create API endpoint and simple page for repository connection.	app/api/routes_repositories.py; frontend/repositories/	Code + test + demo evidence

Week 3 - Code Analysis

End product: a complete analysis report for a Python repository.

Member	Task for this week	Files / artifacts	Done when
M1	Extract Git commit metadata and changed files.	app/analyzers/git_history_analyzer.py	Code + test + demo evidence
M2	Implement AST extraction for functions, classes, imports, and calls.	app/analyzers/ast_parser.py	Code + test + demo evidence
M3	Run pytest-cov and map coverage to files/functions.	app/analyzers/coverage_analyzer.py; app/analyzers/complexity_analyzer.py	Code + test + demo evidence
M4	Create complexity calculation and analysis fixtures.	tests/fixtures/analysis/; app/analyzers/analysis_context.py	Code + test + demo evidence
M5	Detect database, HTTP, filesystem, and external-service calls.	app/analyzers/dependency_detector.py; app/analyzers/api_detector.py	Code + test + demo evidence
M6	Run analysis steps inside the sandbox.	app/services/analysis_service.py; workers/analysis_tasks.py	Code + test + demo evidence
M7	Test analyzer outputs on the selected sample repositories.	tests/unit/analyzers/; tests/integration/analyzer/	Code + test + demo evidence
M8	Store and expose analysis results through the API.	app/models/analysis.py; app/schemas/analysis.py; app/api/routes_analysis.py	Code + test + demo evidence

Week 4 - Risk Ranking

End product: deterministic ranked list of risky functions.

Member	Task for this week	Files / artifacts	Done when
M1	Calculate change-frequency features from Git history.	app/analyzers/git_history_analyzer.py	Code + test + demo evidence
M2	Improve AST relationships between functions and modules.	app/analyzers/call_graph.py; app/models/code_entity.py	Code + test + demo evidence
M3	Implement coverage-gap and complexity features.	app/services/risk_service.py; app/risk/features.py	Code + test + demo evidence
M4	Design risk explanation text and AI explanation prompt.	app/ai/risk_explainer.py; prompts/risk_explanation.txt	Code + test + demo evidence
M5	Define rules for mapping targets to Unit/Integration/API tests.	app/routing/test_type_router.py; app/routing/rules.py	Code + test + demo evidence
M6	Run ranking as a background job in Docker.	workers/risk_tasks.py; app/orchestration/orchestrator.py	Code + test + demo evidence
M7	Write ranking unit tests and repeatability checks.	tests/unit/test_risk_service.py; tests/integration/test_ranking.py	Code + test + demo evidence
M8	Display ranked targets and factor explanations in the UI/API.	app/api/routes_risk.py; frontend/risk/	Code + test + demo evidence

Week 5 - AI Unit Test Generation

End product: generated pytest tests for selected high-risk functions.

Member	Task for this week	Files / artifacts	Done when
M1	Provide source files and target metadata to the generation pipeline.	app/ai/context_builder.py; app/ai/source_context.py	Code + test + demo evidence
M2	Build a reliable AST-to-context serializer.	app/ai/serializers/ast_serializer.py	Code + test + demo evidence
M3	Add coverage gaps and risk reasons to the prompt context.	app/ai/prompt_builder.py; prompts/unit_test.txt	Code + test + demo evidence
M4	Implement Unit Test Agent and structured test output.	app/agents/unit_test_agent.py; app/agents/base_agent.py	Code + test + demo evidence
M5	Collect existing fixtures, helpers, and nearby tests for context.	app/ai/fixture_collector.py; app/ai/test_style.py	Code + test + demo evidence
M6	Save generated tests in a temporary sandbox workspace.	app/services/generation_service.py; workers/generation_tasks.py	Code + test + demo evidence
M7	Validate generated code format and create failure categories.	app/verification/generated_code_validator.py	Code + test + demo evidence
M8	Add Generate Test action and test preview endpoint/page.	app/api/routes_generation.py; frontend/generated-tests/	Code + test + demo evidence

Week 6 - Assumptions and Basic Verification

End product: generated tests are reviewed with evidence and execution results.

Member	Task for this week	Files / artifacts	Done when
M1	Implement temporary test-file and artifact management.	app/services/artifact_service.py; storage/generated/	Code + test + demo evidence
M2	Link generated assertions to source context where possible.	app/ai/assumption_extractor.py; app/ai/assertion_sources.py	Code + test + demo evidence
M3	Measure coverage contribution of each generated test.	app/analyzers/test_coverage_delta.py	Code + test + demo evidence
M4	Implement assumption extraction and Needs Review status.	app/ai/assumption_extractor.py; prompts/assumptions.txt	Code + test + demo evidence
M5	Review generated tests for integration/API boundary cases.	app/agents/integration_test_agent.py; app/agents/api_test_agent.py	Code + test + demo evidence
M6	Implement compile/import checks and pytest execution.	app/verification/compile_checker.py; app/execution/pytest_runner.py	Code + test + demo evidence
M7	Implement repeated runs and stable/unstable status.	app/verification/stability_checker.py; tests/unit/verification/	Code + test + demo evidence
M8	Display test code, assumptions, logs, and verification status.	app/api/routes_verification.py; frontend/verification/	Code + test + demo evidence

Week 7 - Regression and Mutation

End product: tests are evaluated for real bug-catching value.

Member	Task for this week	Files / artifacts	Done when
M1	Implement safe temporary branch/workspace handling.	app/execution/worktree_manager.py; app/services/repository_connector.py	Code + test + demo evidence
M2	Select mutation targets from AST and high-risk functions.	app/verification/mutation_target_selector.py; app/verification/ast_mutator.py	Code + test + demo evidence
M3	Compare coverage before and after generated tests.	app/analyzers/coverage_analyzer.py; app/analyzers/coverage_delta.py	Code + test + demo evidence
M4	Use surviving-mutant feedback to request stronger tests.	app/ai/test_repair.py; prompts/repair.txt	Code + test + demo evidence
M5	Define mutation categories for logic and boundary bugs.	app/verification/mutation_operators.py	Code + test + demo evidence
M6	Run mutants inside isolated Docker jobs.	app/verification/mutation_runner.py; docker/sandbox/mutation.sh	Code + test + demo evidence
M7	Implement killed/survived results and mutation score.	app/verification/mutation_score.py; app/models/mutation.py	Code + test + demo evidence
M8	Add mutation results to the verification report.	app/api/routes_verification.py; frontend/mutation/	Code + test + demo evidence

Week 8 - Integration and API Tests

End product: first multi-level testing prototype.

Member	Task for this week	Files / artifacts	Done when
M1	Prepare repository environment files and test branches.	app/services/repository_connector.py; docker/test-environments/	Code + test + demo evidence
M2	Improve dependency/call relationships for target selection.	app/analyzers/dependency_graph.py	Code + test + demo evidence
M3	Analyze external calls and integration coverage.	app/analyzers/external_call_detector.py; app/analyzers/integration_coverage.py	Code + test + demo evidence
M4	Add integration scenarios to the AI context.	app/ai/integration_context.py; prompts/integration.txt	Code + test + demo evidence
M5	Implement Integration and API Test Agent prototypes.	app/agents/integration_test_agent.py; app/agents/api_test_agent.py	Code + test + demo evidence
M6	Create Docker Compose test database/mock-service environment.	docker-compose.test.yml; docker/test-environments/	Code + test + demo evidence
M7	Run and verify integration/API tests with pytest.	app/verification/integration_runner.py; tests/integration/	Code + test + demo evidence
M8	Add test-type selection and results to the dashboard.	app/routing/test_type_router.py; app/api/routes_tests.py; frontend/test-types/	Code + test + demo evidence

Week 9 - Approval and Git Output

End product: user-controlled test acceptance and branch/patch output.

Member	Task for this week	Files / artifacts	Done when
M1	Implement accepted-test branch creation and commit preparation.	app/services/git_output_service.py; app/services/branch_service.py	Code + test + demo evidence
M2	Check imports and target references before approval output.	app/verification/target_reference_checker.py	Code + test + demo evidence
M3	Show quality metrics before the approval decision.	frontend/metrics/; app/schemas/quality.py	Code + test + demo evidence
M4	Implement regeneration after user feedback.	app/services/regeneration_service.py; app/ai/feedback_context.py	Code + test + demo evidence
M5	Review approval behavior for Integration/API tests.	tests/integration/approval/; app/approval/test_approval_rules.py	Code + test + demo evidence
M6	Run a final sandbox verification before writing output.	app/approval/final_verification.py; workers/approval_tasks.py	Code + test + demo evidence
M7	Run complete verification again after accepted tests are added.	app/verification/post_approval_runner.py	Code + test + demo evidence
M8	Implement Accept/Reject/Edit and approval history UI/API.	app/api/routes_approval.py; app/models/approval.py; frontend/approval/	Code + test + demo evidence

Week 10 - Benchmark Evaluation

End product: reproducible evaluation on BugsInPy/Tests4Py.

Member	Task for this week	Files / artifacts	Done when
M1	Implement benchmark checkout adapter for buggy/fixed subjects.	app/evaluation/bugsinpy_adapter.py; app/evaluation/checkout.py	Code + test + demo evidence
M2	Analyze benchmark subjects with the normal analyzer.	app/evaluation/benchmark_analyzer.py	Code + test + demo evidence
M3	Prepare metric schemas and result tables.	app/evaluation/metrics_schema.py; reports/templates/	Code + test + demo evidence
M4	Run generated tests on buggy and fixed versions.	app/evaluation/benchmark_runner.py; workers/evaluation_tasks.py	Code + test + demo evidence
M5	Classify detected bugs by type and test level.	app/evaluation/bug_classifier.py; app/evaluation/test_level.py	Code + test + demo evidence
M6	Execute benchmark subjects reproducibly in Docker.	docker/benchmark/Dockerfile; docker/benchmark/run_subject.sh	Code + test + demo evidence
M7	Calculate detection rate, mutation score, and coverage change.	app/evaluation/metrics.py; app/evaluation/statistics.py	Code + test + demo evidence
M8	Create evaluation report and benchmark dashboard view.	app/api/routes_evaluation.py; frontend/evaluation/; reports/	Code + test + demo evidence

Week 11 - Integration and Hardening

End product: complete end-to-end demo with error handling.

Member	Task for this week	Files / artifacts	Done when
M1	Test the full Git-to-branch workflow and fix integration issues.	tests/e2e/test_git_workflow.py; docs/git-workflow.md	Code + test + demo evidence
M2	Fix AST edge cases and unsupported syntax handling.	tests/unit/analyzers/test_edge_cases.py; app/analyzers/compatibility.py	Code + test + demo evidence
M3	Validate risk ranking against known benchmark targets.	reports/risk_validation.csv; docs/risk-validation.md	Code + test + demo evidence
M4	Improve prompts, reduce hallucinated imports, and handle repair loops.	prompts/; app/ai/guardrails.py	Code + test + demo evidence
M5	Stabilize integration/API scenarios and mocks.	tests/integration/test_api_and_integration.py; docker/test-environments/	Code + test + demo evidence
M6	Test timeouts, resource limits, dependency failures, and isolation.	tests/security/test_sandbox_limits.py; docker/sandbox/limits.py	Code + test + demo evidence
M7	Run end-to-end verification and regression suite.	tests/e2e/test_full_verification.py; tests/regression/	Code + test + demo evidence
M8	Connect all features into one demo flow and track remaining bugs.	app/orchestration/orchestrator.py; frontend/demo/; docs/demo-script.md	Code + test + demo evidence

Week 12 - Final Delivery

End product: final release, report, presentation, and live demonstration.

Member	Task for this week	Files / artifacts	Done when
M1	Finalize Git integration documentation.	docs/git-integration.md; app/services/repository_connector.py	Code + test + demo evidence
M2	Finalize code-analysis documentation and examples.	docs/analysis.md; app/analyzers/README.md	Code + test + demo evidence
M3	Document risk formula, metrics, and limitations.	docs/metrics.md; docs/limitations.md	Code + test + demo evidence
M4	Document AI prompts, assumptions, and generation limitations.	docs/ai-generation.md; docs/prompts.md	Code + test + demo evidence
M5	Document Unit/Integration/API test behavior.	docs/testing-levels.md; docs/integration-testing.md	Code + test + demo evidence
M6	Document sandbox security and deployment instructions.	docs/sandbox-security.md; deployment/README.md	Code + test + demo evidence
M7	Finalize verification and mutation-testing results.	docs/verification.md; docs/mutation-testing.md	Code + test + demo evidence
M8	Prepare final dashboard demo, report, slides, and release checklist.	docs/final-demo.md; docs/final-checklist.md; presentation/	Code + test + demo evidence